Idea de aplicación: EuroFibo Clock

EuroFibo Clock es una aplicación que selecciona una capital europea utilizando números de Fibonacci y la hora actual de Colombia.

La aplicación mantiene un catálogo de países y capitales europeas, por ejemplo:

```
None
EuropeanCapital("France", "Paris")
EuropeanCapital("Germany", "Berlin")
EuropeanCapital("Spain", "Madrid")
EuropeanCapital("Italy", "Rome")
EuropeanCapital("Portugal", "Lisbon")
```

Sobre este catálogo pueden existir diferentes algoritmos de selección. Cada algoritmo representa una regla de negocio distinta para decidir cuál será la **capital europea del momento**.

Fibonacci se implementa de forma iterativa mediante recursión de cola en A1, A2, A3 y cualquier algoritmo posterior que construyamos.

Es decir, cuando escribamos `Fibonacci(n)`, conceptualmente estaremos hablando de una implementación del estilo:

```
None
fibonacci(n)
    → fibonacciTail(n, 0, 1)

fibonacciTail(n, a, b)
    si n == 0:
        retornar a

    retornar fibonacciTail(
        n - 1,
        b,
        a + b
    )
```

La idea es que los acumuladores `a` y `b` transportan el estado necesario, evitando la forma recursiva clásica que recalcula valores.

Podemos describirlo siempre como:

“Cálculo de Fibonacci mediante recursión de cola, con comportamiento iterativo basado en acumuladores.”

Algoritmo A1 — Fibonacci Colombian Capital Selector

El algoritmo **A1** recibe como entrada un número `n` que indica qué número de Fibonacci se desea computar.

Supongamos que el usuario solicita computar Fibonacci para el número:

```
None
n = 5
```

Después de realizar el cálculo de Fibonacci, se toma el último número relevante producido por dicho cálculo. Supongamos, para este ejemplo, que dicho valor es:

```
None
2
```

El valor solicitado inicialmente y el valor obtenido mediante Fibonacci se suman:

```
None
5 + 2 = 7
```

Este `7` se convierte en el **factor Fibonacci** del algoritmo.

A continuación, la aplicación consulta la hora actual de Colombia mediante una función como:

```
None
getColombianTime()
```

Supongamos que en ese momento la función devuelve:

```
None
13:40:34
```

El algoritmo separa las tres partes de la hora:

```
None
horas    = 13
minutos  = 40
segundos = 34
```

y las suma:

```
None
13 + 40 + 34 = 87
```

El valor 87 representa el **factor temporal colombiano**.

Después, el factor temporal se divide entre el factor Fibonacci:

```
None
87 / 7 = 12
```

Se utiliza únicamente la parte entera:

```
None
12
```

Ese número representa la posición lógica que se recorrerá dentro del catálogo de capitales europeas.

Como el catálogo puede ser más pequeño que el número obtenido, la lista se recorre de manera circular.

Con las siguientes capitales:

```
None
0 → Paris
1 → Berlin
2 → Madrid
3 → Rome
4 → Lisbon
```

el recorrido para el valor 12 sería:

```
None
0  → Paris
1  → Berlin
2  → Madrid
3  → Rome
4  → Lisbon
5  → Paris
6  → Berlin
7  → Madrid
8  → Rome
9  → Lisbon
10 → Paris
11 → Berlin
12 → Madrid
```

Por lo tanto, el resultado de esta ejecución del algoritmo A1 sería:

```
None
EuropeanCapital("Spain", "Madrid")
```

Matemáticamente, el recorrido circular puede representarse como:

```
None
capitalIndex = 12 % 5
capitalIndex = 2
```

y la capital ubicada en el índice 2 es:

```
None
Madrid
```

De esta manera, A1 combina una entrada del usuario, un cálculo de Fibonacci y la hora real de Colombia para **producir una capital europea de forma determinista**.

Algoritmo A2 — Fibonacci Temporal Passport

El algoritmo **A2** representa una regla de negocio diferente.

En este caso, la aplicación interpreta cada ejecución como la creación de un **pasaporte temporal europeo**. El usuario introduce un número y el sistema utiliza Fibonacci junto con la hora colombiana para determinar qué país le corresponde visitar simbólicamente.

Supongamos nuevamente:

```
None
n = 5
```

Primero se calcula:

```
None
Fibonacci(5)
```

Supongamos que el resultado utilizado por la implementación es:

```
None
5
```

Ahora se construye un **código Fibonacci** sumando el número solicitado y el resultado:

```
None
5 + 5 = 10
```

Después se consulta la hora colombiana:

```
None
13:40:34
```

Pero A2 no suma simplemente sus componentes. Cada parte de la hora tiene un significado dentro del dominio:

```
None
hora     → coordenada de salida
minuto   → distancia temporal
segundo  → factor de variación
```

Primero se multiplica la hora colombiana por el código Fibonacci:

```
None
13 × 10 = 130
```

Luego se suman los minutos:

```
None
130 + 40 = 170
```

Finalmente, los segundos se utilizan como factor de variación:

```
None
170 + 34 = 204
```

El resultado es el **European Temporal Score**:

```
None
ETS = 204
```

Ahora se utiliza el número de capitales disponibles para normalizar el resultado:

```
None
capitalIndex = ETS % cantidadDeCapitales
```

Como existen cinco capitales:

```
None
capitalIndex = 204 % 5
capitalIndex = 4
```

Por lo tanto:

```
None
4 → Lisbon
```

El resultado de A2 sería:

```
None
EuropeanCapital("Portugal", "Lisbon")
```

Además de seleccionar la capital, A2 genera un identificador de pasaporte:

```
None
FIB-5-COL-134034-204-LISBON
```

El resultado completo del dominio podría ser:

```
None
TemporalPassport(
    fibonacciInput = 5,
    fibonacciValue = 5,
    colombianTime = "13:40:34",
    temporalScore = 204,
    destination = EuropeanCapital("Portugal", "Lisbon"),
    passportCode = "FIB-5-COL-134034-204-LISBON"
)
```

Así, mientras **A1** funciona como un selector circular basado en una división entre el tiempo colombiano y Fibonacci, **A2** funciona como un generador de “pasaportes temporales”, donde Fibonacci actúa como multiplicador de la hora colombiana para determinar matemáticamente un destino europeo.

Esto permite que ambos algoritmos formen parte del mismo dominio:

```
None
CapitalSelectionAlgorithm
 ├── A1: Fibonacci Colombian Capital Selector
 └── A2: Fibonacci Temporal Passport
```

y que en el futuro puedan agregarse nuevos algoritmos `A3`, `A4`, etc., cada uno con una regla distinta para combinar Fibonacci, tiempo colombiano y capitales europeas.

Algoritmo A3 — Viajero Temporal Europeo

El algoritmo **A3** representa un modelo de negocio en el que la aplicación actúa como un **oráculo de viaje europeo**.

La idea principal es combinar dos elementos:

- Fibonacci determina **desde qué capital comienza el viaje**.
- La hora colombiana determina **cuántas posiciones debe avanzar el viajero** dentro de una lista circular de capitales europeas.

El cálculo de Fibonacci se realiza, como en los demás algoritmos, mediante una implementación basada en **recursión de cola con acumuladores**, manteniendo un comportamiento iterativo.

La aplicación dispone de la siguiente lista circular de capitales:

```
None
0 → Paris
1 → Berlin
2 → Madrid
3 → Rome
4 → Lisbon
5 → Vienna
6 → Prague
7 → Warsaw
8 → Budapest
9 → Athens
```

Paso 1 — Fibonacci determina el punto de partida

El usuario introduce un número:

```
None
n = 5
```

Se calcula Fibonacci utilizando recursión de cola:

```
None
Fibonacci(5) = 5
```

El resultado de Fibonacci se utiliza para determinar la posición inicial del viaje.

Como la lista tiene 10 capitales, la posición inicial se normaliza mediante módulo:

```
None
posiciónInicial = Fibonacci(n) % cantidadDeCapitales
```

Para este ejemplo:

```
None
posiciónInicial = 5 % 10
posiciónInicial = 5
```

La capital ubicada en la posición `5` es:

```
None
Vienna
```

Por lo tanto, el viaje comienza en **Vienna**.

Paso 2 — La hora colombiana determina el desplazamiento

A continuación, la aplicación obtiene la hora actual de Colombia.

Supongamos que la función devuelve:

```
None
13:40:34
```

A3 interpreta cada componente como parte de una fórmula de desplazamiento.

Primero se multiplican las horas por los minutos y posteriormente se suman los segundos:

```
None
desplazamientoTemporal =
    hora × minuto + segundo
```

Aplicando los valores del ejemplo:

```
None
13 × 40 + 34
```

```
None
520 + 34 = 554
```

Por lo tanto:

```
None
desplazamientoTemporal = 554
```

Como existen únicamente 10 capitales, el desplazamiento también debe normalizarse utilizando módulo:

```
None
desplazamiento =
    desplazamientoTemporal % cantidadDeCapitales
```

Entonces:

```
None
554 % 10 = 4
```

Esto significa que el viajero debe avanzar **4 posiciones** desde su capital inicial.

Paso 3 — Recorrer Europa de forma circular

El viajero comienza en:

```
None
5 → Vienna
```

y avanza cuatro posiciones:

```
None
6 → Prague
7 → Warsaw
8 → Budapest
9 → Athens
```

Por lo tanto, el destino final recomendado por A3 es:

```
None
Athens
```

Matemáticamente, la posición final también puede calcularse directamente:

```
None
posiciónFinal =
    (posiciónInicial + desplazamiento)
    % cantidadDeCapitales
```

Para este caso:

```
None
posiciónFinal =
    (5 + 4) % 10
```

```
None
posiciónFinal = 9
```

Y la posición 9 corresponde a:

```
None
EuropeanCapital("Greece", "Athens")
```

Resultado del algoritmo

El resultado de A3 no representa únicamente una capital seleccionada, sino un pequeño recorrido de viaje.

Conceptualmente podría representarse como:

```
None
EuropeanJourney(
    fibonacciInput = 5,
    fibonacciValue = 5,
    startCapital = Vienna,
    colombianTime = "13:40:34",
    temporalDisplacement = 554,
    effectiveDisplacement = 4,
    destinationCapital = Athens
)
```

De esta manera, **A3 utiliza Fibonacci como coordenada inicial y la hora colombiana como fuerza de desplazamiento**, construyendo un viaje virtual a través de una secuencia circular de capitales europeas.

La pregunta de negocio que responde A3 puede expresarse como:

> **¿Desde qué capital comienza tu viaje europeo según Fibonacci y cuántas ciudades debes avanzar según la hora actual de Colombia?**

No separaría A1, A2 y A3 en dominios distintos. Los tres pertenecen al mismo dominio porque comparten el mismo lenguaje y los mismos conceptos centrales: Fibonacci, hora colombiana, capitales europeas y una regla que produce un resultado de viaje. De hecho, ya planteamos que sobre el mismo catálogo pueden existir distintos algoritmos, cada uno como una regla de negocio diferente.

La separación que haría es esta: **un solo dominio**, pero con **estrategias de negocio distintas** para A1, A2 y A3.